CS661: Big Data Visual Analytics
Project Proposal

Group-1

Praveen (240790) Virendra Kala (241172) Utkarsh Aman (241114)
Vishakha Sharma (241173) Anand Sachan (240116) Ranisha (240850)
Yash Dabi (241195) Arnab Patra (240186) Akshita (240090)

---

1 Introduction

Open-source software is everywhere today – the tools developers use daily like React, TensorFlow, and Kubernetes are all community-maintained projects. But how do you actually know if a project is healthy? Is it still being actively maintained? Are bug reports getting answered? Is the whole project depending on just one or two people?

Right now there is no easy way to answer these questions without manually going through GitHub page by page, which is slow and doesn't let you compare projects side by side. Our project aims to build a web-based interactive visual analytics system that helps users understand the health and contributor activity of open-source GitHub repositories. We want to cover things like how fast pull requests get merged, how quickly maintainers respond to issues, whether a project has a bus-factor problem, and how activity levels have changed over time.

We have already verified that the data is accessible – GH Archive (`https://www.gharchive.org/`) records every public GitHub event since 2011 and is hosted as a free public dataset on Google BigQuery. We ran a few test queries on repositories like `facebook/react` and it works well.

2 Data Source

Our primary data source is the GH Archive dataset available on Google BigQuery. It contains structured JSON records of all public GitHub events (pull requests, issues, commits, etc.) with hourly granularity since 2011. We can query it directly with SQL, and up to 1TB/month is free.

For this project we will focus on:

- **30–50 repositories** from three areas: frontend frameworks (React, Vue, Svelte), ML/data tools (TensorFlow, PyTorch, scikit-learn), and backend/DevOps tools (Django, FastAPI, Kubernetes).
- **Time range:** January 2023 to December 2024, which gives us two years of history without making the data too large to work with.
- **Event types:** PullRequestEvent, IssuesEvent, IssueCommentEvent, PushEvent, WatchEvent, and ReleaseEvent.

We will pull this data using the BigQuery Python client, store it locally in SQLite, and compute derived features (like PR latency or bus-factor scores) from there.

3 Specific Tasks

For this project, we will perform the following main tasks:

- **Data Extraction:** Query the GH Archive dataset on BigQuery and pull relevant events for the selected repositories into a structured local format.
- **Data Preprocessing:** Clean and process the raw event data by handling missing values, removing inconsistencies, and transforming it into a form suitable for analysis.
- **Feature Engineering:** Derive higher-level metrics from the raw data such as PR review latency, bus-factor scores, issue response times, and bot vs. human activity ratios.
- **Data Visualization:** Build interactive visual representations of repository health metrics, contributor patterns, and activity trends.
- **User Interaction:** Integrate interactive features that allow users to filter by repository, time range, or ecosystem, and have all panels update together in real time.

4 Visualization Tasks

We plan to implement six visualization tasks. Each one targets a specific question about open-source project health, and all panels will be linked so that filtering in one updates the others.

4.1 Technology Adoption Trends

Which ecosystems are growing or declining over time?

This visualization will show how overall activity volume changes across repositories and ecosystems over the two-year period. Users will be able to:

- Filter by ecosystem (frontend, ML, backend) to focus on a specific group.
- Compare activity levels across multiple repositories over time.
- Identify periods of sudden growth or decline in a project.

Suggested visualization: Streamgraph or stacked area chart showing monthly activity, with options to filter by ecosystem or repository.

4.2 Contributor Collaboration Network

Who is actually doing the work, and is there a bus-factor risk?

This visualization will help understand how contributors interact with each other – who reviews whose code, who collaborates frequently, and whether the project is overly dependent on a small group of people. We will also compute a bus-factor score to quantify this risk. Users will be able to:

- Select a repository and see its contributor network.
- Identify highly central contributors whose absence could be a risk.
- Filter by time period to see how the network has evolved.

Suggested visualization: Force-directed network graph or adjacency matrix, with node size or color indicating contribution level.

4.3 Repository Health Summary

How does one repository compare to another at a glance?

This will be a summary panel that brings together key health metrics for any selected repository in one place, and also supports side-by-side comparison of two repositories. Users will be able to:

- View key metrics like median merge time, bus-factor score, issue response rate, and bot activity percentage.
- Compare two repositories directly side by side.
- See how metrics update live when filters are applied elsewhere in the system.

Suggested visualization: KPI cards with sparklines, or a radar/spider chart for multi-metric comparison across repositories.

4.4 Pull Request Lifecycle and Review Latency

Where does the code review process slow down?

This visualization will show how pull requests move through different stages – from being opened, to first review, to approval, to being merged or closed – and how long each stage takes. Users will be able to:

- Select a repository and see where PRs tend to get stuck or dropped.
- Compare review latency across different repositories.
- Filter by time period to see if things have improved or gotten worse.

Suggested visualization: Sankey diagram showing PR flow through stages, paired with box plots or violin plots for time-to-merge distributions.

4.5 Bot vs. Human Activity

How much of the activity is real, and how much is just automation?

Many GitHub projects have bots handling tasks like dependency updates and CI checks. This visualization will break down how much of a project's activity is bot-generated vs. human-generated, and allow users to filter bots out from other panels so the metrics stay meaningful. Users will be able to:

- See the proportion of bot vs. human activity per repository.
- Toggle bots on/off to recompute metrics across all panels.
- Compare bot usage patterns across different ecosystems.

Suggested visualization: Stacked bar chart or pie chart breaking down event types by bot vs. human, with a global filter toggle.

4.6 Issue Responsiveness

Is the project actively responding to bug reports and questions?

This visualization will show how consistently a project responds to issues over time, making it easy to spot periods where maintainers were inactive or overwhelmed. Users will be able to:

- View issue activity over the two-year period for any repository.
- Spot gaps or irregular silences in maintainer responses.
- Compare responsiveness across different repositories.

Suggested visualization: Calendar heatmap or time-series line chart showing daily issue open and close activity.

5 Overall Solution

The system will be a single-page web application built using Plotly Dash. All six panels will be cross-linked, meaning selecting a date range or clicking on a repository in any panel will update all others in real time. The idea is that a user could, for example, click a repository in the network graph and immediately see how its PR latency, issue responsiveness, and health metrics compare – without having to navigate anywhere. The real value is in being able to drill down from a high-level trend to the specific data behind it.

6 Tech Stack

- **Data Extraction:** Google BigQuery Python client
- **Data Processing:** Python, Pandas, NumPy, NetworkX
- **Database:** SQLite
- **Visualization and App:** Plotly, Dash, Dash-Cytoscape

7 Team Members and Responsibilities

- **Data Pipeline** (BigQuery queries, SQLite storage): Ranisha (240850), Akshita (240090)
- **Feature Engineering** (PR latency computation, bus-factor, bot detection): Utkarsh Aman (241114), Vishakha Sharma (241173)
- **Network Analysis** (contributor graph, community detection): Anand Sachan (240116)
- **Visualization Development** (Dash app, all six panels): Praveen (240790),Virendra Kala (241172)
- **Integration, Testing, and Report:** Arnab Patra (240186), 241195 (Yash Dabi)

This project aims to use big data analytics and visualization techniques to better understand the health and activity patterns of open-source software projects. Our system will serve as a useful tool for developers, project maintainers, and researchers to explore GitHub repository data interactively and make more informed decisions about the tools they use or contribute to.